Crypto

```
#Simple Caesar Cipher function - brute force approach with a plaintext and a given shift


def caesarCipher(plainText, shift):
    cipherText = ""
    for i in range(len(plainText)):
        if plainText[i].isupper():
            cipherText += chr((ord(plainText[i]) + shift - 65) % 26 + 65) #65 is the ASCII value of 'A' and 26 is the number of letters in the alphabet
        else:
            cipherText += chr((ord(plainText[i]) + shift - 97) % 26 + 97) #97 is the ASCII value of 'a' and 26 is the number of letters in the alphabet
    return cipherText


#Driver code
print(caesarCipher("Hello World", 5))


#Caesar Cipher Decoding function - brute force approach with a ciphertext and a given shift
def caesarCipherDecoder(cipherText, shift):
    plainText = ""
    for i in range(len(cipherText)):
        if cipherText[i].isupper():
            plainText += chr((ord(cipherText[i]) - shift - 65) % 26 + 65) #65 is the ASCII value of 'A' and 26 is the number of letters in the alphabet
        else:
            plainText += chr((ord(cipherText[i]) - shift - 97) % 26 + 97) #97 is the ASCII value of 'a' and 26 is the number of letters in the alphabet
    return plainText
```

```
#Base64 Decoder function - useful in many instances
import base64

def base64Decoder(cipherText):
  plainText = base64.b64decode(cipherText).decode('utf-8', errors='ignore')
  return plainText

#Driver code
print(base64Decoder("SGVsbG8gV29ybGQ="))

#Vigenere Cipher function - brute force approach with a plaintext and a given key, modulo the key length also
def vigenereCipher(plainText, key):
  cipherText = ""
  for i in range(len(plainText)):
    if plainText[i].isupper():
      cipherText += chr((ord(plainText[i]) + ord(key[i % len(key)]) - 65) % 26 + 65) #65 is the ASCII value of 'A' and 26 is the number of letters in the alphabet
    else:
      cipherText += chr((ord(plainText[i]) + ord(key[i % len(key)]) - 97) % 26 + 97) #97 is the ASCII value of 'a' and 26 is the number of letters in the alphabet
  return cipherText

#Driver code
print(vigenereCipher("Hello World", "key"))

#Vigenere Cipher Decoder function - brute force approach with a ciphertext and a given key, modulo the key length also
def vigenereCipherDecoder(cipherText, key):
  plainText = ""
```

```
  for i in range(len(cipherText)):
    if cipherText[i].isupper():
      plainText += chr((ord(cipherText[i]) - ord(key[i % len(key)]) - 65) % 26 + 65) #65 is the ASCII value of 'A' and 26 is the number of letters in the alphabet
    else:
      plainText += chr((ord(cipherText[i]) - ord(key[i % len(key)]) - 97) % 26 + 97) #97 is the ASCII value of 'a' and 26 is the number of letters in the alphabet
  return plainText

#Frequency Analysis function - useful when we have no information about the cipher text
def frequencyAnalysis(cipherText):
  freq = {}
  for i in cipherText:
    if i in freq:
      freq[i] += 1
    else:
      freq[i] = 1
  return freq

#Driver code
print(frequencyAnalysis("Hello World"))

#DES Encyption and Decryption Function - useful to implement the DES given a text, a key and a mode
#Steps are implemented as follows:
#Rule 1. Choose a plaintext that is divisible into 12bit 'blocks'
#Rule 2. Choose a key at least 8bits in length
#Rule 3. For each block from i=0 while i<N perform the following operations
#Rule 4. Repeat the following operations on block i, from r=0 while r<R
```

#Rule 5. Divide the block into 2 6bit sections Lr,Rr

#Rule 6. Using Rr, "expand" the value from 6bits to 8bits.

Do this by remapping the values using their index, e.g.

1 2 3 4 5 6 -> 1 2 4 3 4 3 5 6

#Rule 7. XOR the result of this with 8bits of the Key beginning with Key[iR+r] and wrapping back to the

beginning if necessary.

#Rule 8. Divide the result into 2 4bit sections S1, S2

#Rule 9. Calculate the 2 3bit values using the two "S boxes" below, using S1 and S2 as input respectively.

#

S1 0 1 2 3 4 5 6 7

0 101 010 001 110 011 100 111 000

1 001 100 110 010 000 111 101 011

#

S2 0 1 2 3 4 5 6 7

0 100 000 110 101 111 001 011 010

1 101 011 000 111 110 010 001 100

#Rule10. Concatenate the results of the S-boxes into 1 6bit value

#Rule11. XOR the result with Lr

#Rule12. Use Rr as Lr and your altered Rr (result of previous step) as Rr for any further computation on block i

#Rule13 increment r

#We do use many auxiliary functions to implement the DES

#then we can see both the encryption and decryption functions

Binary to text and text to binary

def string2binary(text):

```
    return ''.join(f'{ord(c):08b}' for c in text)

def binary2string(text):
    return ''.join(f'{ord(c):08b}' for c in text)

# Split into 2 6bit blocks
def splitblock(block):
    Lr = block[:6]
    Rr = block[6:]
    return Lr, Rr

# Expand the block from 6 to 8 bits
def expand_miniblock(b):
    return b[0] + b[1] + b[3] + b[2] + b[3] + b[2] + b[4] + b[5]

# XOR function between two binary strings and return the result as 8 bit binary string
def xor(a, b):
    res = int(a, 2) ^ int(b, 2)
    return f'{res:08b}'

# Create S1 and S2
def rule9S1(a):
    row = int(a[0]) # 0 or 1 (first bit)
    col = int(a[1:], 2) # 2 is the base and [1:] means from the second element to the end

    matrix = [['101','010', '001', '110', '011', '100', '111', '000'],
        ['001', '100',  '110',  '010',  '000','111',    '101',  '011']
    ]
```

```
    return matrix[row][col]


def rule9S2(a):
    row = int(a[0]) # 0 or 1 (first bit)
    col = int(a[1:], 2) # 2 is the base and [1:] means from the second element to the end

    matrix = [['100','000','110','101','111','001','011','010'],
        ['101','011','000','111','110','010','001','100']]

    return matrix[row][col]

# Encryption Function
def encrypt(text, key, R):
    text_encr = ''

    text_bin = string2binary(text)
    if (len(text_bin) % 12 ) != 0:
        raise Exception('Error in rule 1')

    key_bin = string2binary(key)
    if (len(key_bin) < 8):
        raise Exception('Error in rule 2')

    for bnum in range(len(text_bin) // 12): # 12 bits per block
        i = bnum # index of the block

        from_   = 0 + 12*bnum # start of the block (12bit)
        to_     = 12*(bnum+1) # end of the block (12bit), so it's bnum+1
        block   = text_bin[from_:to_] # get the block from the text
```

```
    for r in range(R): # R rounds
        Lr, Rr = splitblock(block) # split the block in 2 6bit blocks

        Rr_expanded = expand_miniblock(Rr) # expand the 6bit block to 8bit

        curr_key = key_bin[i * R + r : i * R + r + 8] # get the key for the current round (8bit)

        Rr_exp_xor_key = xor(Rr_expanded, curr_key) # XOR between the expanded block and the key

        S1 = Rr_exp_xor_key[:4] # get the first 4 bits
        S2 = Rr_exp_xor_key[4:] # get the last 4 bits

        S1update = rule9S1(S1) # apply rule 9 to S1, updating accordingly
        S2update = rule9S2(S2) # apply rule 9 to S2, updating accordingly

        S = S1update + S2update # concatenate the 2 updated blocks
        if len(S) != 6: # check if the length is 6
            raise Exception('Error in rule 10')

        newRr = xor(S, Lr)[2:] # XOR between the S block and the Lr block, and get the last 4 bits

        block = Rr + newRr # concatenate the Rr and the newRr
    text_encr += block # add the block to the encrypted text
  return text_encr # return the encrypted text

# Decrypt Function
def decrypt(text, key, R):
```

```
    text_bin = text
    key_bin = string2binary(key)
    text_dec = ''

    if len(text_bin) < 8:
        raise Exception('Error in rule 2')

    for bnum in range(len(text_bin) // 12):
        i = bnum # index of the block

        from_   = 0 + 12*bnum  # start of the block (12bit)
        to_     = 12*(bnum+1)  # end of the block (12bit), so it's bnum+1
        block   = text_bin[from_:to_] # get the block from the text

        for r in range(R-1, -1, -1): # R rounds but we do reverse the previous loop
            Rr, Rr_alt = splitblock(block) # split the block in 2 6bit blocks

            Rr_expanded = expand_miniblock(Rr) # expand the 6bit block to 8bit

            curr_key = key_bin[i * R + r : i * R + r + 8] # get the key for the current round (8bit)
            Rr_exp_xor_key = xor(Rr_expanded, curr_key) # XOR between the expanded block and the key

            S1 = Rr_exp_xor_key[:4] # get the first 4 bits
            S2 = Rr_exp_xor_key[4:] # get the last 4 bits

            S1update = rule9S1(S1) # apply rule 9 to S1, updating accordingly
            S2update = rule9S2(S2) # apply rule 9 to S2, updating accordingly
```

```
        S = S1update + S2update # concatenate the 2 updated blocks
        if len(S) != 6: # check if the length is 6
            raise Exception('Error in rule 10')

        Lr = xor(Rr_alt, S) # XOR between the Rr_alt block and the S block
        Lr = Lr[2:] # get the last 4 bits
        block = Lr + Rr # concatenate the Lr and the Rr

    new = Lr + Rr # concatenate the Lr and the Rr
    text_dec += new # add the block to the decrypted text

res = ''

# Convert the binary to a string, simply taking each 8 bytes of text, saw it in binary
# then convert it to a character, wrapping it all back to a string

for i in range(len(text_dec) // 8): # 8 bits per block
    res += chr(int(text_dec[(i*8): ((i+1)*8)], 2)) # convert the binary string to a character
print(res) # print the decrypted text

key = 'mu' # key
R = 2 # number of rounds

en = encrypt('Min0n!', key, R) # encrypt the text
decrypt(en, key, R) # decrypt the text

#MD5 Hashing Algorithm
import hashlib
def md5hashing(text):
```

```
    hash_object = hashlib.md5(text.encode())
    print(hash_object.hexdigest())

#Driver Code
md5hashing('Min0n!')

#MD5 Reverse Hashing Algorithm - It probably won't work, cause its a bruteforce
#of all the possible combinations of characters on a hash, so it can be an infinite runtime
import hashlib
def md5reversehashing(text):
    for i in range(0, 100000000): # 100000000 is the number of possible combinations
        hash_object = hashlib.md5(str(i).encode()) # convert the number to a string and encode it
        if hash_object.hexdigest() == text: # if the hash is equal to the text
            print(i) # print the number
            break
#Driver Code
md5reversehashing('e10adc3949ba59abbe56e057f20f883e') # 123456
#md5reversehashing('365d38c60c4e98ca5ca6dbc02d396e53') # This one coming from the challenges is not working

#XOR Function Between 2 Numbers - They must be of the same length
def xor(a, b):
    if len(a) != len(b):
        raise Exception('Error in rule 1')
    return ''.join([str(int(a[i]) ^ int(b[i])) for i in range(len(a))])

#Driver code
print(xor('1010', '0101')) # 1111
#Remember that the XOR function is commutative, so the order doesn't matter
```

```
#Also, we can't XOR strings, so we have to convert them to binary


#XOR example between two words (already converted)
#key = ' '.join([chr(ord(x) ^ ord(y)) for x, y in zip(original_data, enc_data)])


#Opening a file (no matter the extension) and reading it
def openfile(filename):
   with open(filename, 'r') as file: #alternatively, we can use 'rb' for binary files
      data = file.read().replace('\n', '') # read the file and replace the new line character with nothing
   return data


#Driver Code
#openfile('test.txt')
```